

Individual Tree Segmentation Using Deep Learning and Climbing Algorithm: A Method for Achieving High-precision Single-tree Segmentation in High-density Forests under Complex Environments

He Ma, Fangmin Zhang, Simin Chen, and Jinge Yu

Abstract

Accurate individual tree segmentation, which is important for forestry investigation, is still a difficult and challenging task. In this study, we developed a climbing algorithm and combined it with a deep learning model to extract forests and achieve individual tree segmentation using lidar point clouds. We tested the algorithm on mixed forests within complex environments scanned by unmanned aircraft system lidar in ecological restoration mining areas along the Yangtze River of China. Quantitative assessments of the segmentation results showed that the forest extraction achieved a kappa coefficient of 0.88, and the individual tree segmentation results achieved F-scores ranging from 0.86 to 1. The climbing algorithm successfully reduced false positives and false negatives with the increased crown overlapping and outperformed the widely used top-down region-growing point cloud segmentation method. The results indicate that the climbing algorithm proposed in this study will help solve the overlapped crown problem of tree segmentation under complex environments.

Introduction

Forest ecosystems are complex functional systems, occupying one-third of the Earth's land area (Schiefer *et al.* 2020). As the largest carbon reservoir in terrestrial ecosystems, forests play a crucial role in maintaining the global carbon balance, significantly affecting future global climate stability and the development of human society (Torabzadeh *et al.* 2019). To accurately analyze the detailed conditions of global forests, efficient and accurate methods for forest inventory are urgently needed to effectively depict the detailed structure and current status of forests. Individual tree-based surveys, compared to regional statistical surveys, can more precisely obtain important parameters such as timber volume, biomass, and carbon storage, aiding in the accurate quantitative analysis of forests.

The key prerequisite for individual tree-based forest surveys is the high-precision extraction of individual trees. Only by accurately determining the information of individual trees in a forest can subsequent forest parameter extraction be performed. Therefore, the precise extraction of individual trees using remote sensing methods is essential

for large-scale, detailed forest surveys, aiding in forest management and optimization. Lidar, as an emerging remote sensing technology, provides data with higher resolution and three-dimensional point cloud data, which can depict more detailed characteristics of individual trees and forest biomass (Li *et al.* 2015; Zhao *et al.* 2009). With technological advancements, airborne lidar scanning (ALS) increasingly provides extensive information on forest stands or tree characteristics (average tree height, tree count, and individual tree height) (Lovell *et al.* 2011), but it often lacks detailed information about the branch layer (Dassot *et al.* 2011). Additionally, compared to terrestrial lidar scanning (TLS), ALS typically has a lower point density, often limited to 10 points/m² (Lu *et al.* 2014). However, with the miniaturization of sensors and the rapid development of technologies related to unmanned aircraft systems (UAS), UAS lidar scanning (ULS) has achieved a point density that is 2 orders of magnitude higher than ALS (Kellner *et al.* 2019), making it feasible to extract individual tree information over large forest areas.

In the past two decades, individual tree segmentation algorithms have undergone rapid development. These methods can be roughly divided into two major categories (Zhen *et al.* 2016). The traditional category includes raster-based methods that use the canopy height model (CHM) as foundational data. In most cases, CHM is selected to identify local maxima at tree-canopy tops and then delineate crown areas around the maxima by using methods such as watershed segmentation (Wang *et al.* 2004), valley following (Gougeon 1995), edge detection (Koch *et al.* 2006), morphological reconstruction (Liu *et al.* 2016), or template matching (Pirotti 2010). However, existing studies have shown that CHM-based methods struggle to detect trees in the lower forest layers (Eysn *et al.* 2015). Moreover, broadleaf trees are predominantly asymmetrical in canopy form. The CHM approach produces poorer results when compared to methods that work directly from point clouds (Jaskierniak *et al.* 2015).

With technological innovation, more advanced tree detection techniques that operate directly on point clouds have been developed. One of the most well-known and commonly used methods is the top-down region-growing point cloud segmentation (PCS) method developed by Li *et al.* (2012). PCS achieved an overall accuracy of 94% in the Sierra National Forest, significantly surpassing traditional marker-controlled watershed segmentation (Tao *et al.* 2014). PCS has gradually become the mainstream individual tree segmentation method for ALS and ULS point cloud data. Despite this, PCS and other top-down segmentation methods have seen slow progress in improving accuracy.

He Ma, Fangmin Zhang and Jinge Yu are with the Collaborative Innovation Center on Forecast and Evaluation of Meteorological Disasters/Jiangsu Key Laboratory of Agricultural Meteorology, Nanjing University of Information Science and Technology, (fmin.zhang@nuist.edu.cn).

Simin Chen is with the School of Arts, Nanjing University of Information Science and Technology.

Corresponding author: Fangmin Zhang (fmin.zhang@nuist.edu.cn)

Received July 15, 2024, accepted October 17, 2024.

Photogrammetric Engineering & Remote Sensing
Vol. 91, No. 2, February 2025, pp. 101–110.
0099-1112/22/101–110

© 2024 American Society for Photogrammetry
and Remote Sensing
doi: 10.14358/PERS.24-00083R2

In recent years, more research begun to explore bottom-up segmentation methods such as the comparative shortest-path (CSP) algorithm (Tao *et al.* 2015) and individual tree and crown delineation (Jaskierniak *et al.* 2021). With the swift advancement of deep learning, the idea of combining deep learning with individual tree segmentation has gained increasing attention. Windrim and Bryson (2020) proposed a new deep learning-based method for detecting and segmenting individual trees as well as reconstructing tree trunks, achieving an accuracy of 92% and demonstrating that deep learning can aid in single-tree segmentation tasks. However, bottom-up segmentation methods often require TLS or other sources capable of acquiring complete forest point cloud data, including the branches point cloud below the canopy. Such complete data are usually not available from ALS and ULS.

Previous studies often focused on plantations or natural forests with relatively homogeneous tree species (Balsi *et al.* 2018; Wallace *et al.* 2014; Yin and Wang 2019). However, it is noteworthy that benchmark studies by Eysn *et al.* (2015) evaluated eight tree-detection algorithms and found that forest structure significantly affects the results, which is consistent with the findings of Vauhkonen *et al.* (2012). Both of these studies indicate that there still exist significant challenges in extracting forests and further segmenting individual trees in areas with diverse understory types and highly overlapping tree branches. Moreover, ULS data often fail to capture point clouds below the canopy, resulting in incomplete point clouds for most trees, which poses additional difficulties for individual tree segmentation.

To address the challenge of individual tree segmentation within complex environments, we proposed a new individual tree segmentation algorithm called the “climbing algorithm.” We combined it with the PointMLP deep learning model to test its performance on individual tree segmentation with tree crown overlapping under complex environments based on 200 regional samples along the Yangtze River in Jiangsu Province, China. This research will provide new ideas and solutions for individual tree segmentation in forestry surveys and management processes.

Methods

In the following sections, first the data used is introduced, and then the technical methods for achieving individual tree segmentation are described with the specific process illustrated in Figure 1. The process is divided into three steps: first, preprocessing the point cloud data; then, using a deep learning architecture to perform semantic segmentation on the target point cloud; and finally, constructing a new individual tree segmentation algorithm based on the forest extraction results of the deep learning semantic segmentation, namely the climbing algorithm, to achieve individual tree segmentation. Parameter information involved in the implementation of the entire scheme is presented, and the relevant indicators and methods for verifying the accuracy are introduced subsequently.

Data Sets

The lidar data for this study were collected from 200 regional samples along the Yangtze River in Jiangsu Province, China, in June 2023 using the Huace AA10 and AU20 laser aerial survey systems. The ranging accuracy, repeatability, and system accuracy of the Huace AA10 and AU20 systems are 15 mm, 5 mm, and 5 cm, respectively. More detailed parameters of the Huace AA10 and AU20 systems are shown in Table 1. Figure 2 shows the distribution of the study area, as well as the remote sensing image and lidar data of one regional sample. To ensure that the deep learning model can achieve relatively accurate results, we additionally selected the publicly available ALS-based Dayton Annotated Laser Earth Scan (DALES) point cloud data set from the University of Dayton (Singer and Asari 2021) and the publicly available individual tree point cloud data set based on ALS and ULS collected in mixed forests in Germany (Weiser *et al.* 2022) as supplements to the deep learning training and testing sets.

Preprocessing

Because the deep learning model has specific input standards, a series of preprocessing steps must be performed on the field-collected lidar

point cloud data. First, the point cloud is normalized using the following formula:

$$Z_{iNORM} = Z_i - Z_i^{DEM} \quad (1)$$

where Z_{iNORM} is the normalized Z value of the i th point, Z_i is the Z value of the i th point, and Z_i^{DEM} is the corresponding value of the digital elevation model (DEM) at that point's location. After normalization, the elevation value of each point represents the height of that point above the ground. This process only adjusts the height of the point cloud without affecting its overall structure.

Next, the point cloud data is denoised using the following two methods:

$$P_{valid} = \{p_i \in P | Z_{p_i} < Z_{max}\} \quad (2)$$

$$P_{valid} = \{p_i \in P | \exists p_j \in P, \text{dist}(p_i, p_j) < r\} \quad (3)$$

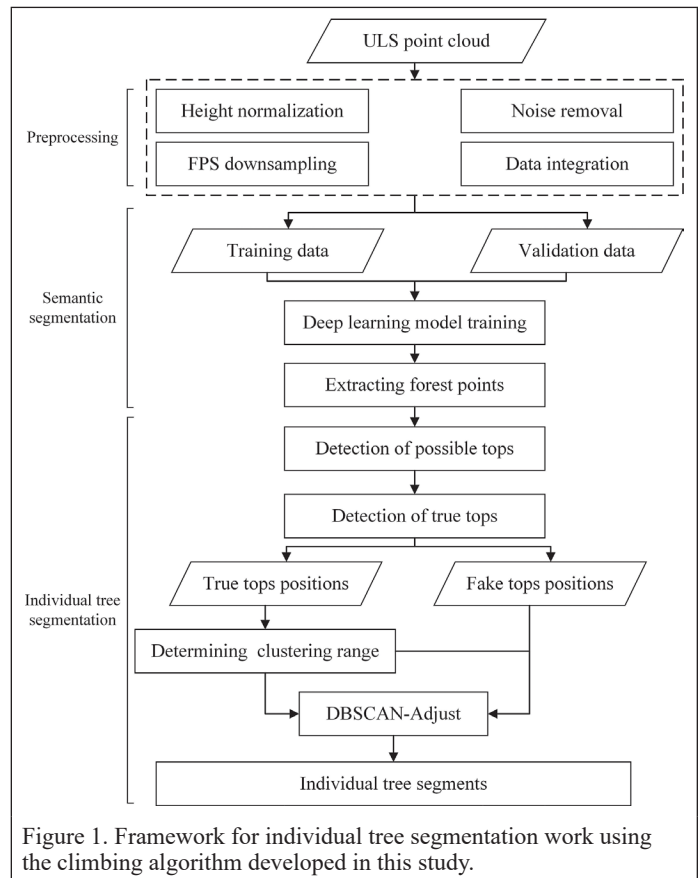


Figure 1. Framework for individual tree segmentation work using the climbing algorithm developed in this study.

Table 1. Specifications of AA10 and AU20 lidar systems used for scanning samples.

AA10	Ranging range	10 to 800 m
	Ranging accuracy/repeatability	15 mm/5 mm
	Maximum pulse rate	500 000 points/second
	Scan line speed	50 to 250 lines/second
	Field of view	75°
	System accuracy	5 cm (horizontal), 5 cm (vertical)
AU20	Ranging range	1.5 to 1500 m
	Ranging accuracy/repeatability	15 mm/5 mm
	Maximum pulse rate	2 000 000 points/second
	Scan line speed	10 to 200 lines/second
	Field of view	0° to 360°
	System accuracy	5 cm (horizontal), 5 cm (vertical)

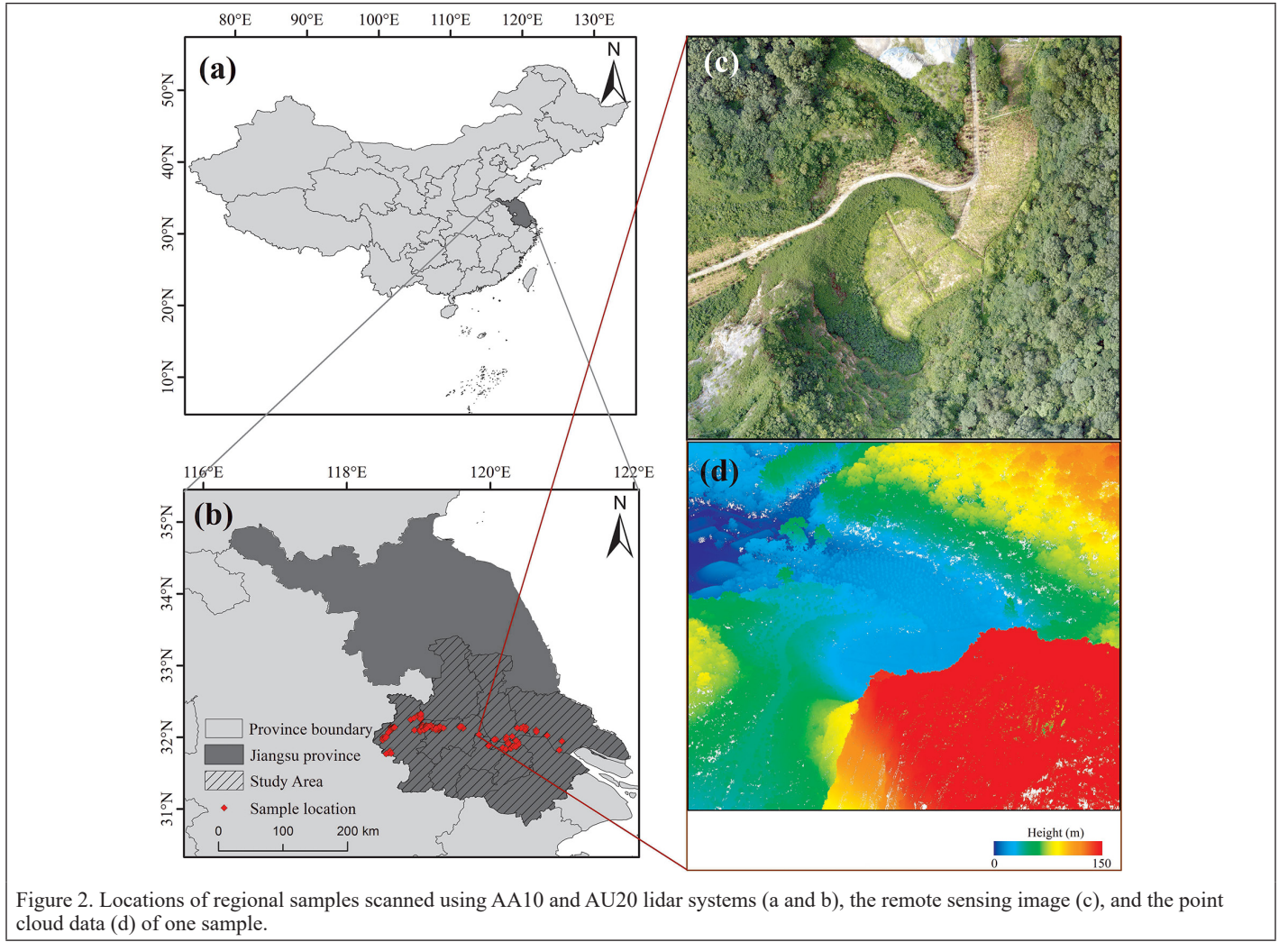


Figure 2. Locations of regional samples scanned using AA10 and AU20 lidar systems (a and b), the remote sensing image (c), and the point cloud data (d) of one sample.

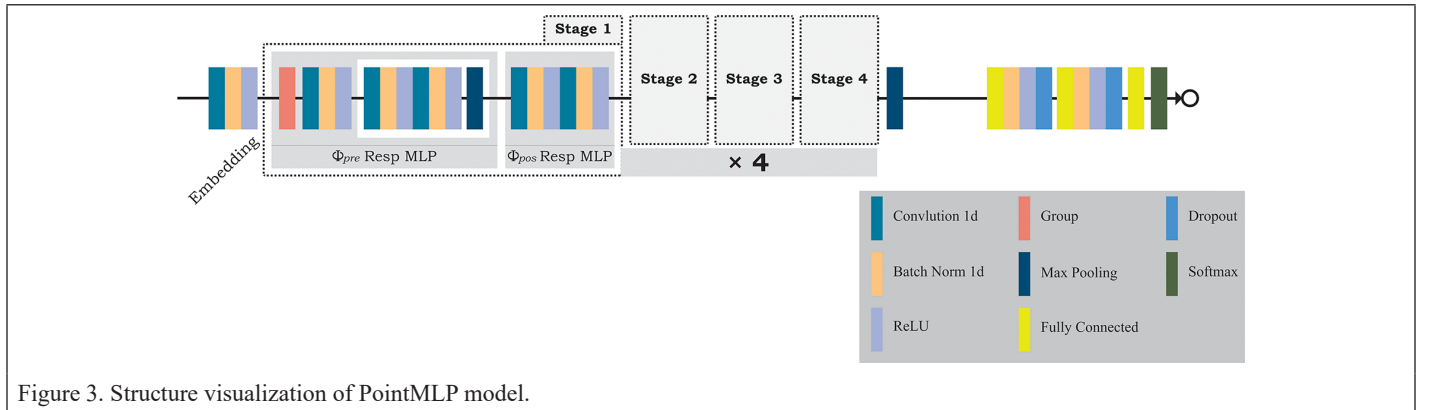


Figure 3. Structure visualization of PointMLP model.

where P_{valid} is the point cloud data after denoising, Z_{max} is the height threshold of the point cloud data, r is the radius threshold of the isolated point, and $\text{dist}(p_i, p_j)$ is the distance between p_i and p_j . Equation 2 is used to remove outliers with abnormally high values, and Equation 3 is used to remove isolated points that are not clearly connected to any point cloud clusters. After preprocessing, we selected point cloud data from 16 sample areas, manually segmented the forests, buildings, roads, and farmland within them, and assigned labels to these categories. These labeled data were combined with representative point cloud data from existing public data sets. Then the farthest point sampling method was used to ensure that the number of points in each point cloud meets the input standards of the PointMLP model. The integrated data set consists of 848 building point clouds, 2426 forest point clouds,

330 farmland point clouds, and 227 road point clouds. Finally, 70% of the data were randomly selected as the training set, and 30% were reserved as the testing set.

PointMLP Deep Learning Method

To accurately extract forest point clouds from a complex point cloud environment, the PointMLP deep learning model was selected as the method for forest semantic segmentation. PointMLP is a deep residual multilayer perceptron (MLP) network that adopts a design philosophy similar to PointNet and PointNet++, using a simpler but deeper network architecture (Ma *et al.* 2022). As shown in Figure 3, PointMLP uses a prefeedback residual MLP network to progressively aggregate local features extracted by MLPs, without requiring any fine-grained local feature extractors. Therefore, this method can avoid the extensive

computational workload and continuous memory access caused by complex local feature extraction. Additionally, a lightweight geometric affine module is introduced to adaptively transform local points into a normal distribution, further enhancing the model's performance and generalization ability. The structure of the model is shown in Figure 3, and the core algorithm of the network can be represented by the following formula:

$$g_i = \Phi_{\text{pos}}(A[\Phi_{\text{pre}}(f_{ij}), |j = 1, \dots, K]) \quad (4)$$

where g_i is the output feature of the i^{th} point, Φ_{pre} and Φ_{pos} are residual MLP modules, Φ_{pre} is used to learn shared weights from the local region, and Φ_{pos} is used to extract deeply aggregated features. The aggregation function $A[\]$ is a max-pooling function. The model uses the k -nearest neighbor algorithm to select nearby points, where $k = 24$. PointMLP uses a four-layer deep network, and such a computation process needs to be repeated four times.

PointNet model independently learns point features using multiple MLP layers and extracts global features using a max-pooling layer. As the early proposed point cloud recognition depth model, it is widely used and is the basis for all subsequent MLP class models (Liu *et al.* 2020; Qi *et al.* 2017); therefore, the PointNet model was selected for comparisons with the PointMLP model in this study. The model contains three main modules: a symmetric function that aggregates information from all points, a combination of local and global information, and a joint alignment network. The global features extracted by the model are classified using a MLP classifier. The PointNet model can be represented by Equation 5:

$$f(x_1, x_2, \dots, x_n) = \gamma(\text{MAX}_{i=1, \dots, n} \{h(x_i)\}) \quad (5)$$

where $(x_1, x_2, \dots, x_n)$ is the set of input points to the model, $\gamma(\)$ and $h(\)$ are multilayer perceptron networks, and the function $f(\)$ is invariant to the permutation of the input points.

Individual Tree Segmentations

Because the ULS forest point clouds cannot record most trunk information below the canopy in dense forest areas, a deep learning model such as PointNet or PointMLP can only provide poor results when executing instance segmentation. To solve this problem, a new method called the “climbing algorithm” was developed. This climbing algorithm achieves individual tree segmentation by identifying

tree crown vertices and performing density clustering downward from these vertices.

Typically, although crown overlapping is common in dense forest areas, there is still some spacing between trees at the canopy height. We assume that the crown of each individual tree in a dense forest area can reach the average canopy height. Based on this assumption, we first focus on searching for all possible tree vertices. As shown in Figure 4a, we set a possible vertex search radius R_p and traversed all points. If there exists a point such that within a radius R_p centered on it, there are no points with a higher Z value; therefore, this point can be considered a possible vertex. However, as illustrated in Figure 4a, the canopy shape of individual trees is often irregular, and a tree's crown may consist of multiple branch groups. Taking this into account, directly considering possible vertices as true vertices would result in many false positives. To overcome this issue, we set a minimum tree spacing, R_t . As shown in Figure 4b, starting from the possible vertex with the lowest Z value, if within a radius R_t centered on this point, there is no other possible vertex with a greater Z value, then this possible vertex is considered a true vertex, and the search is repeated for the next lowest Z value point. If the next point has other any vertex with a greater Z value within a radius R_t , it is marked as a false vertex, and the search is repeated for the higher possible vertices found within the range. Eventually, all possible vertices will be traversed and classified as either false vertices or true vertices, and we can obtain the true tree vertex set, T . Through these two steps, we ensure the accuracy of the tree crown vertices while preserving the characteristic information of the tree crowns as much as possible.

After confirming all the true vertices, we used the Density-Based Spatial Clustering of Applications with Noise algorithm (DBSCAN) to perform clustering segmentation of individual trees. Unlike the k -means algorithm, DBSCAN is a density-based clustering algorithm that does not require the number of clusters as an input parameter. Instead, by defining the minimum number of points required to form a cluster (MinPts) and the neighborhood radius (Eps neighborhood of a point), DBSCAN can automatically detect clusters of arbitrary shapes. Starting from any point p in the point cloud D , the Eps neighborhood of p , denoted as $N_{Eps}(p)$, is defined as the set of points q within the distance Eps from point p (including point p itself):

$$N_{Eps}(p) = \{q \in D | \text{dist}(p, q) \leq Eps\} \quad (6)$$

If this neighborhood contains enough points, which means that $|N_{Eps}(q)| \geq \text{MinPts}$, a cluster is initiated; otherwise, p is marked as a

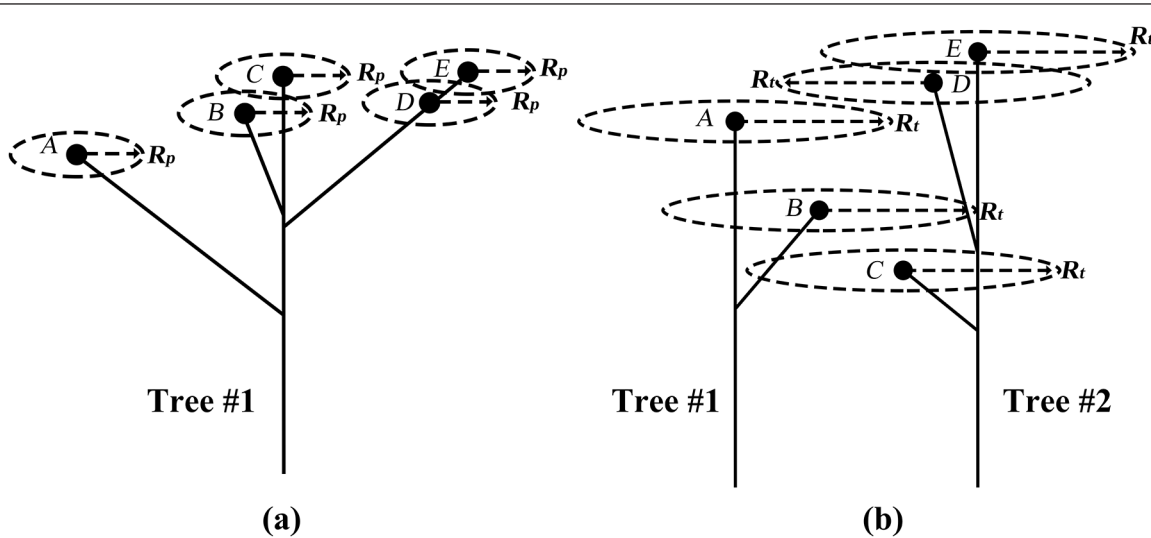


Figure 4. Algorithm for detecting possible tops and true tops. We first detect all possible vertices using R_p and then filter out the true vertices of each tree using R_t . Figure 4(a) shows the detection process of possible vertices, and Figure 4(b) illustrates the filtering process of true vertices. R_p = possible vertex search radius; R_t = minimum tree spacing radius.

noise point (Ester *et al.* 1996; Wu *et al.* 2013). This process is repeated for each unvisited point.

Given that crown overlapping is very common in dense forest areas and that the DBSCAN clustering algorithm does not consider cluster shapes, adjustments are made to the DBSCAN algorithm to prevent overclustering. As shown in Figure 5, we modified its execution logic. First, the starting point for DBSCAN is restricted to true vertices instead of any point. Second, only points within the Eps neighborhood of the current point p with Z values lower than point p are accessed and counted, forming a top-down clustering pattern. Finally, ground points are excluded from participating in the DBSCAN clustering:

$$N_{Eps}(p) = \{q \in D | \text{dist}(p, q) \leq Eps | Z_q < Z_p | Z_q > 0\}, p_0 \in T \quad (7)$$

We call the adjusted algorithm the “DBSCAN-Adjust algorithm.” The purpose of this adjustment is to prevent DBSCAN from overclustering in point clouds with severely overlapped crowns, which could result in multiple trees being classified into the same cluster.

To further prevent the DBSCAN algorithm from overclustering while preserving the characteristic information of each tree as much as possible, we restrict the scope of DBSCAN to ensure that it does not perform clustering operations indefinitely in dense forest areas. Additionally, to address the issue of significant crown overlapping, we divide the scope into two parts: the certain area and the possible area. Points within the certain area can only be clustered once, whereas the possible area often contains a large portion of overlapping trees, where point clouds can be clustered multiple times. In other words, a point in the possible area can simultaneously be part of multiple clusters.

Figure 6 depicts the schematic diagram of area division to define an individual tree. First, we denote the distance between the current true vertex being clustered and its nearest true vertex in the horizontal direction as R_d . Different schemes are adopted to define the maximum

clustering range based on different R_d values. As shown in Figure 6a, if $R_d > 2R_t$, the area within a radius R_t from the true vertex is defined as the certain area, and the annular area within a radius $R_d - R_t$ from the true vertex, excluding the certain area, is defined as the possible area. As shown in Figure 6b, if $R_d < 2R_t$, the area within a radius $R_t - R_d$ from the true vertex is defined as the certain area, and the annular area within a radius R_t from the true vertex, excluding the certain area, is defined as

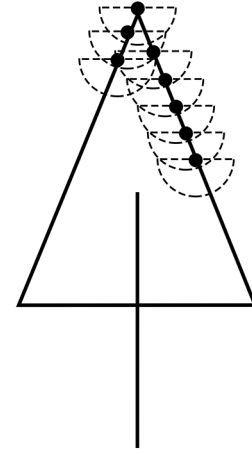


Figure 5. Schematic diagram showing adjustments based on the DBSCAN-Adjust algorithm. The clustering can only start at true points and can only proceed from top to bottom. DBSCAN = Density-Based Spatial Clustering of Applications with Noise

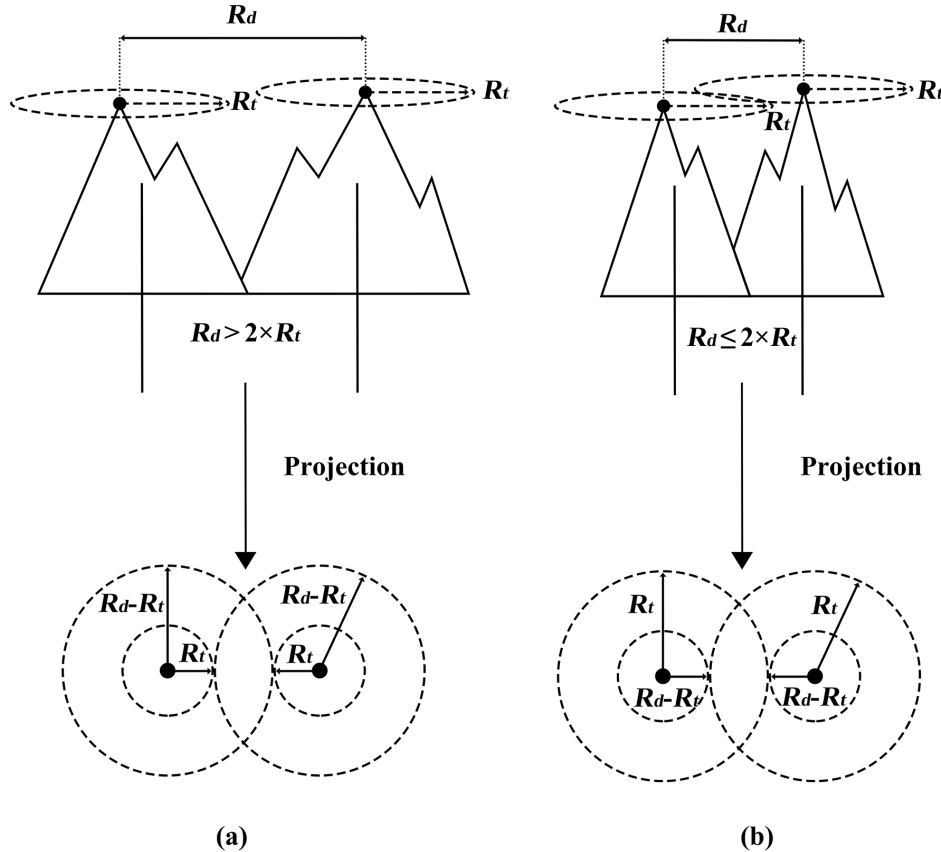


Figure 6. Schematic diagram of area division to define an individual tree. Figure 6(a) shows the method for determining clustering range when trees are spaced far apart, while Figure 6(b) illustrates the method for determining clustering range when trees are spaced closely together. R_d = distance between the current true vertex being clustered and its nearest true vertex in the horizontal direction; R_t = minimum tree spacing radius.

the possible area. The clustering results of each true vertex are confined within these areas, ensuring that overclustering does not result in an excessively large individual tree in dense point cloud regions.

Through the above steps, we can already obtain the segmentation results of individual trees in dense forest areas. On this basis, we can add other steps to achieve more precise individual tree segmentation results. For example, a secondary DBSCAN-Adjust operation can be performed on all false vertices within the range of each individual tree segmentation result to obtain a more complete individual tree point cloud.

Implementation

We used the publicly available data from Ma *et al.* (2022) and used PyTorch (1.8.1 + CUDA 11.1) as the framework for the deep learning method. An NVIDIA GeForce RTX 3070 (8 GB) was used to execute the deep learning training, with the hyperparameters set as shown in Table 2.

Table 2. Summary of hyperparameters for PointMLP and PointNet deep learning models used for obtaining point cloud samples to build forest semantic segmentation.

Hyperparameter	PointMLP Model	PointNet Model
Number of classes	4	4
Number of points	2048	2048
Batch size	32	32
Maximum epoch	150	100
Minimum learning rate	0.005	0.005
Learning rate	0.1	0.1
Weight decay	0.0001	0.0001
Optimizer	SGD	Adam

SGD = Stochastic Gradient Descent; Adam = Adaptive Moment Estimation.

After obtaining the expected training results, to reduce memory resource consumption, we applied the model training results using a sliding window method with a window side length of 5 m to extract the forest parts from complex terrains. Then, we constructed the climbing algorithm, setting the possible vertex search radius R_p to 30 cm, the minimum tree spacing R_t to 150 cm, the Eps in the DBSCAN-Adjust algorithm to 5 cm, and MinPts to 1 for all test point clouds.

Accuracy Assessment

In order to analyze the results more clearly, the following metrics based on Equations 8 to 10 were used to evaluate the model: balanced accuracy (BAcc), precision (Pr), recall (Re), and F-score. BAcc is the arithmetic mean of the recall rates for each category. For the semantic segmentation results, kappa coefficient (κ), overall accuracy (OA), mean class accuracy (mAcc), and mean intersection over union (mIoU), calculated by Equations 11 to 14, were additionally selected as evaluation metrics.

$$Pr = \frac{TP}{TP+FP} \quad (8)$$

$$Re = \frac{TP}{TP+FN} \quad (9)$$

$$F\text{-score} = 2 \times \frac{Pr \times Re}{Pr + Re} \quad (10)$$

$$\kappa = \frac{p_0 - p_e}{1 - p_e} \quad (11)$$

$$OA = \frac{\sum_{i=0}^k TP_i + \sum_{i=0}^k TN_i}{\sum_{i=0}^k (TP_i + FP_i + FN_i + TN_i)} \quad (12)$$

$$mAcc = \frac{1}{k+1} \sum_{i=0}^k \frac{TP_i}{TP_i + FN_i} \quad (13)$$

$$mIoU = \frac{1}{k+1} \sum_{i=0}^k \frac{TP_i}{TP_i + FP_i + FN_i} \quad (14)$$

where p_0 is observed agreement, p_e is chance agreement, TP is true positive, FP is false positive, FN is false negative, and TN is true negative. k is the number of categories involved in the calculation. The F-score from 0 to 1 can be interpreted as the weighted average of precision and recall. The κ value ranging between -1 and 1 is a metric used for consistency testing and can also be used to measure the effectiveness of classification.

Results

We recorded the changes in training accuracy and loss rate during the training process of PointMLP and PointNet models (Figure 7). As shown in Figure 7a, with the increase in training epochs, the training and testing accuracies of the PointMLP model both eventually reached values greater than 0.9, and the loss rate gradually decreased and stabilized with the increasing number of training iterations. As shown in Figure 7b, the PointNet model exhibited a similar trend in accuracy and loss rate changes relative to the PointMLP model, but its accuracy ultimately stabilized around 0.7, which indicates a significant gap compared to the PointMLP model.

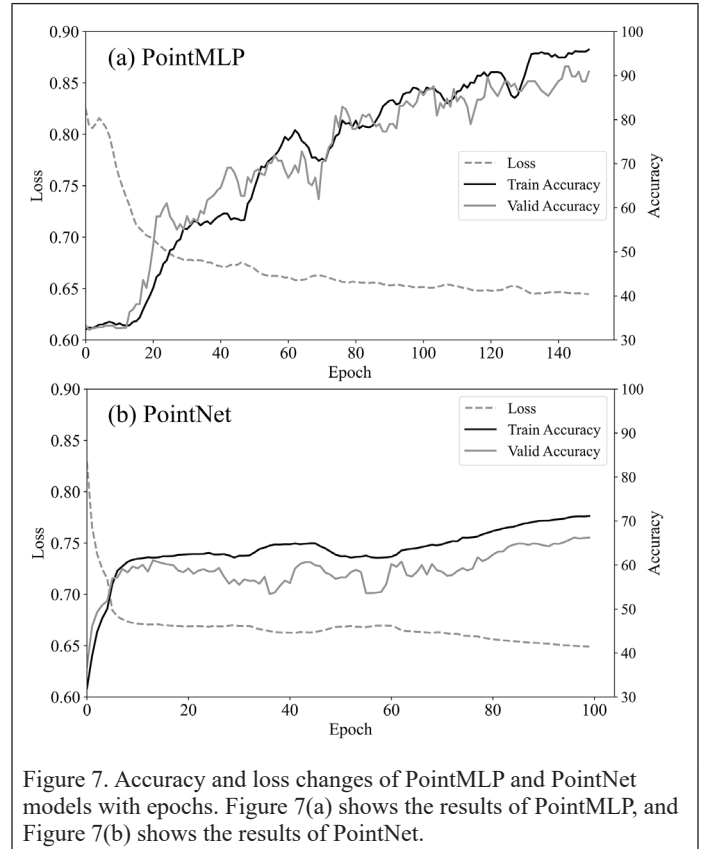


Figure 7. Accuracy and loss changes of PointMLP and PointNet models with epochs. Figure 7(a) shows the results of PointMLP, and Figure 7(b) shows the results of PointNet.

The confusion matrices of the classification results using the two models were described in Figure 8. The PointMLP model demonstrated high accuracy in predicting plot classifications, whereas the PointNet model exhibited more errors in the prediction process. The comparison results showed that evaluation metrics for the PointMLP model are all around 0.9, whereas none of the evaluation metrics for the PointNet model exceeded 0.8 (Table 3). Overall, the PointMLP model has a significant advantage over the PointNet model in terms of classification results.

Figure 9 shows the segmentation results obtained using the climbing algorithm and PCS on three forest sample points. As shown in plot a of Figure 9, it can be observed that in cases where the trees are relatively sparse and there is minimal overlapped crown, both the climbing

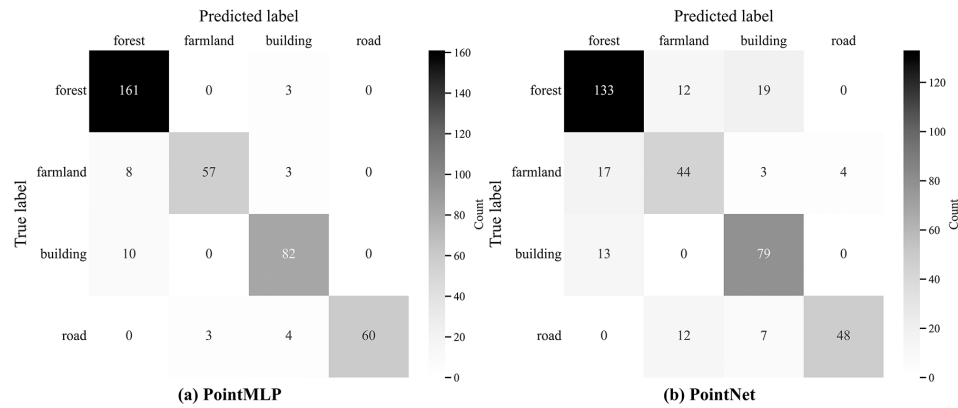


Figure 8. Confusion matrices of the classification results using PointMLP and PointNet models.

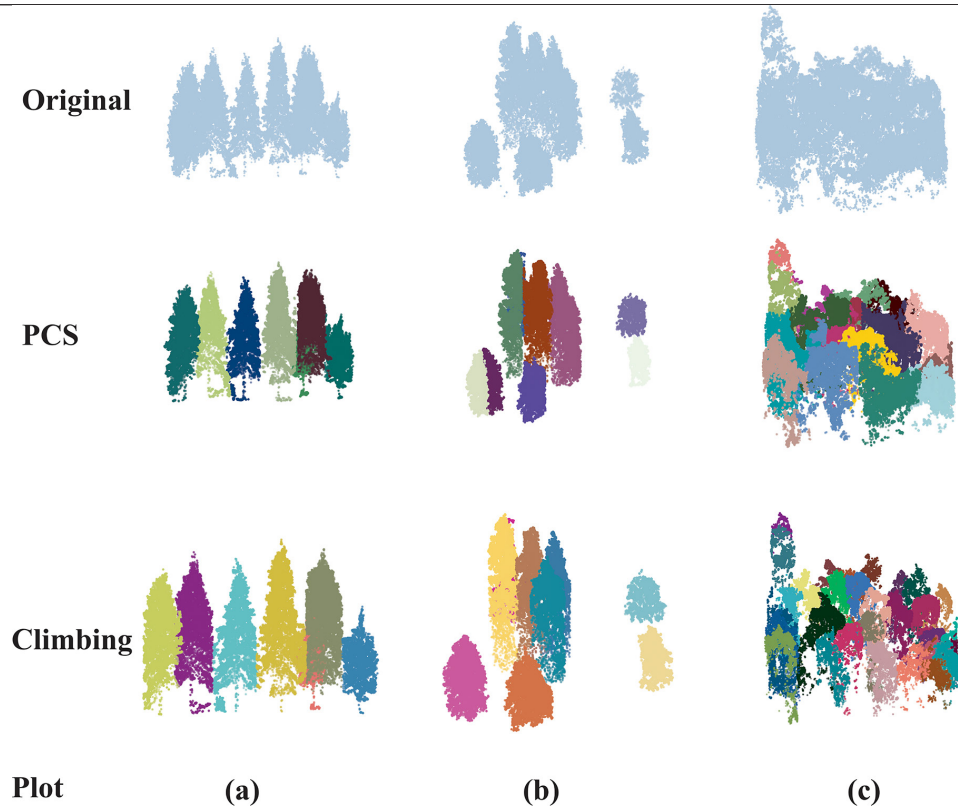


Figure 9. Trees segmented using the point cloud segmentation (PCS) and climbing algorithms for three plots. Different colors indicate different trees. In the climbing algorithm, for points shared by multiple clusters, their color is represented by the color of their first clustering. Figure 9(a), Figure 9(b), and Figure 9(c) show the results of plot a, plot b, and plot c, respectively.

Table 3. Comparisons of evaluation metrics in accuracy assessments of forest extraction using PointMLP and PointNet models.

Evaluation Metrics	PointMLP	PointNet
Pr	0.92	0.78
Re	0.93	0.78
F-score	0.92	0.77
κ	0.88	0.69
BAcc	0.91	0.76
OA	0.92	0.78
mAcc	0.90	0.76
mIoU	0.84	0.61

κ = kappa coefficient; BAcc = arithmetic mean of the recall rates for each category; F-score = weighted average of precision and recall; mAcc = mean class accuracy; mIoU = mean intersection over union; OA = overall accuracy; Pr = precision; Re = recall.

algorithm and PCS accurately achieve individual tree segmentation. Both methods correctly identified six trees without any omissions or false segmentations. As shown in plot b of Figure 9, with increasing crown overlap, PCS successfully segmented only 8 individual trees of 11 with Pr, Re, and F-score of 0.89, 0.73, and 0.8, respectively, whereas the climbing algorithm still maintained accurate identification and segmentation of each individual tree with Pr, Re, and F-score of 1.0, 1.0, and 1.0, respectively. In the case of significant crown overlap and irregular tree growth distribution as in plot c in Figure 9a, both the climbing algorithm and PCS exhibited omissions and false segmentations. According to Table 4, the results using the climbing algorithm for plot c included two false positives and four false negatives. PCS incorrectly segmented trees for plot 3, resulting in six false positives and seven false negatives. In terms of Pr, Re, and F-score, the overall accuracy of the climbing algorithm was still much higher compared with PCS for plot c. This indicates that the climbing algorithm consistently

outperformed PCS with the increasing crown overlap. As shown in Figure 10, the climbing algorithm also consistently performed well when handling more overlapping trees. It is noteworthy that these segmentation results in Figure 10 were based on the same set of parameters used in Figure 9, which demonstrates that the climbing algorithm can maintain accuracy without frequently changing parameters when dealing with different forest densities.

In summary, the climbing algorithm is a more flexible improvement over the PCS algorithm, enabling it to adapt to mixed forests with severe tree branch overlapping and significant variations in tree spacing. The results show that the climbing algorithm can effectively segment individual trees in complex terrains, demonstrating a clear advantage over the PCS algorithm.

Table 4. Comparisons of evaluation metrics in accuracy assessments of segmentation results on the three representative plots in Figure 9 by climbing and PCS algorithms.

Evaluation Metrics	Plot a		Plot b		Plot c	
	PCS	Climbing	PCS	Climbing	PCS	Climbing
TP	6	6	8	11	15	18
FP	0	0	1	0	6	2
FN	0	0	3	0	7	4
Pr	1	1	0.89	1	0.71	0.9
Re	1	1	0.73	1	0.68	0.82
F-score	1	1	0.8	1	0.69	0.86

FN = false negative; FP = false positive; F-score = weighted average of precision and recall; PCS = point cloud segmentation; Pr = precision; Re = recall; TN = true negative; TP = true positive.

Discussion

Comparison of Classifications by PointMLP Model and PointNet Model

In this study, we used the PointMLP model to perform the task of extracting forests in complex environments that include buildings, forests, farmland, cliffs, and other diverse structures and conducted a comparative analysis of its performance against PointNet. The results demonstrated that the PointMLP model exhibited significant performance advantages over the PointNet model in executing the extraction task. This finding is consistent with the conclusions by Liu *et al.* (2022). The difference in performance is mainly attributed to the fact that the PointNet model did not incorporate a local feature extraction module in its design, which limited its ability to classify and recognize similar objects. In contrast, PointMLP significantly improved the model's accuracy by incorporating a local feature extraction module. By comparing the confusion matrices of the classification results of the two models, we found that the classification accuracy for forests was higher for PointMLP, possibly because the number of training samples for forest point clouds was greater than that of other categories. During the training process, we set 2048 sampling points for each sample, which yielded satisfactory results. This aligns with the conclusion of Liu *et al.*, which indicated that training results with 2048 sampling points were superior to those with 1024 sampling points. In summary, the PointMLP model demonstrated excellent performance in the task of extracting forests in complex environments, proving its superiority.

Comparison of Segmentation Result by Climbing Algorithm and PCS Algorithm

In cases where crown overlapping is not significant, both PCS and the climbing algorithm exhibited high accuracy in tree segmentation. As shown in Figure 6a, a situation with uniform spacing between trees and relatively regular shapes of each tree crown is very suitable for the use of the PCS method. PCS identifies local maxima as seed points and

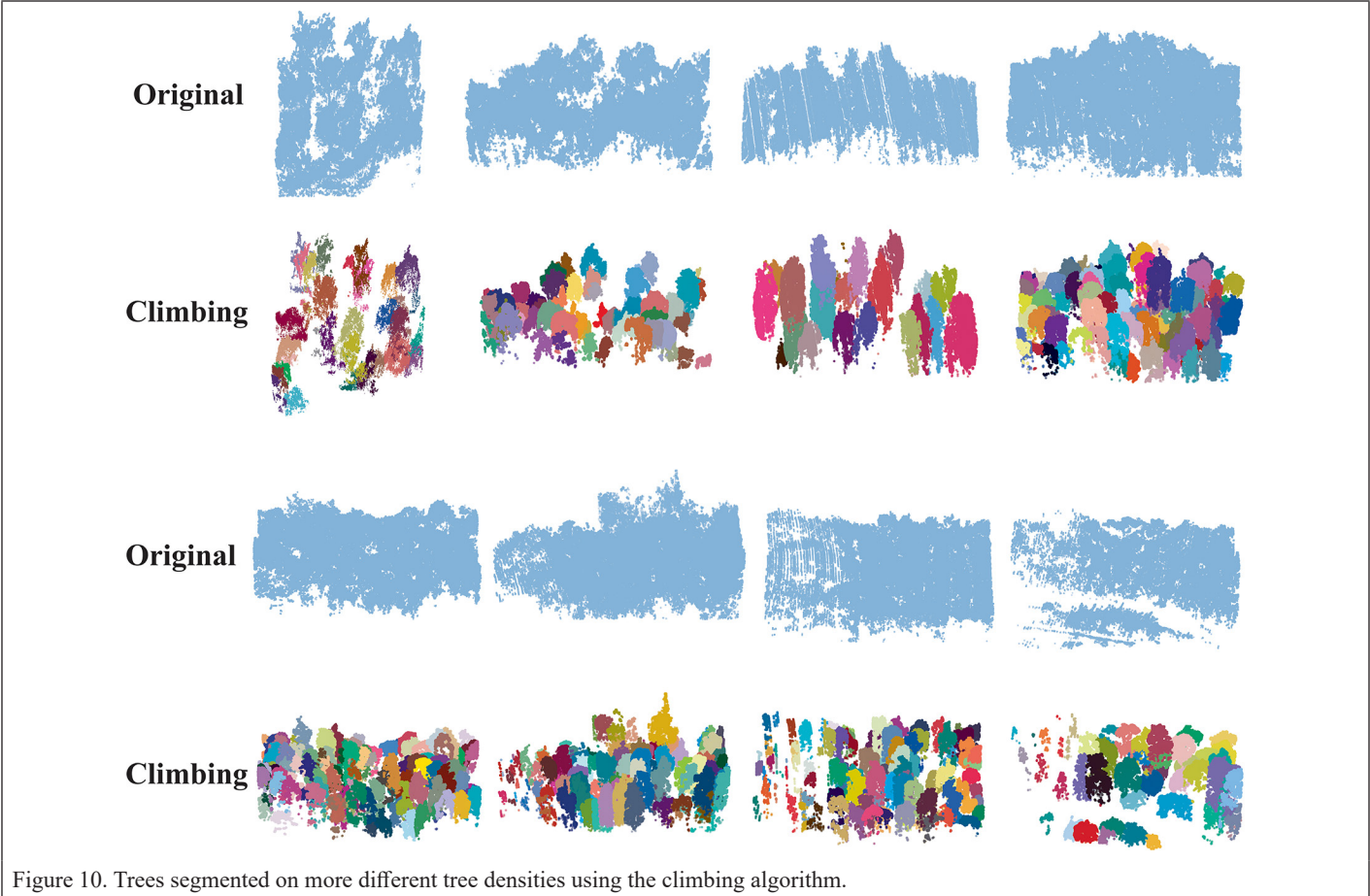


Figure 10. Trees segmented on more different tree densities using the climbing algorithm.

then segments trees by determining a predefined horizontal spacing threshold. A constant spacing value can accurately segment each individual tree, and the conical shape of the tree crowns greatly reduces the likelihood of false positives for seed points.

However, as crown overlapping begins to occur, PCS must select a smaller spacing threshold to accurately segment individual trees in dense areas. Such a choice usually cannot accommodate all tree segmentation tasks. As shown in Figure 6b, PCS set a smaller spacing threshold, which successfully segmented individual trees in dense areas but resulted in false positive identifications for other trees with larger canopy ranges. Conversely, if the spacing threshold is increased, PCS will produce a large number of false negatives in the segmentation of dense tree areas. At this point, the superiority of the climbing algorithm begins to manifest. Because the climbing algorithm does not define the range for each tree as a fixed value but rather analyzes the distance relationship between the current vertex and adjacent vertices, it can adapt well to uneven tree spacing.

From the perspective of preserving individual tree characteristics, the climbing algorithm also demonstrates superior performance as compared to PCS. The climbing algorithm divides the clustering range into certain and possible areas, where points in the possible area can be shared by multiple clusters. This approach better preserves the overlapping parts of the tree crown point clouds, instead of directly segmenting a smooth surface from a fixed spacing threshold. The climbing algorithm first calculates possible vertices and later filters out the true vertices. This method, compared to PCS's direct selection of seed points, better preserves the general characteristics of the trees and, to some extent, avoids missing the vertices of shorter trees during the search. Based on these two features, the climbing algorithm has a clear advantage over PCS.

Parameter Selection of Climbing Algorithm

The climbing algorithm requires the setting of several parameters during its execution. However, these parameters are not very sensitive to the test data set used in this study and can be set to fixed values in most cases. Because the point cloud denoising process has already been completed during data preprocessing, the MinPts parameters in the DBSCAN-Adjust algorithm can be directly set to 1 or a relatively small value. For ULS point cloud data, the point cloud density is relatively uniform in most areas, allowing us to set a constant *Eps* clustering radius for clustering in most study areas. The DBSCAN-Adjust algorithm, based on DBSCAN, not only can discover clusters of arbitrary shapes, but also, unlike the *k*-means clustering method, does not require the number of clusters to be predefined, making it better suited to large-scale forest point clouds.

The climbing algorithm uses two radii, R_d and R_p , to define possible vertices and true vertices. These radii are based on the minimum possible range of tree crowns and individual trees rather than the average spacing, which gives the climbing algorithm a significant accuracy advantage in forest point cloud segmentation tasks with varied tree spacing. The climbing algorithm's approach of selecting the minimum value allows it to maintain a constant radius threshold in large-scale, multi-region segmentation tasks, greatly reducing the workload in large-scale point cloud segmentation.

Limitations with Climbing Algorithm

The climbing algorithm proposed in this study is designed for individual tree segmentation of ULS and ALS point clouds in dense forest areas, following a top-down clustering sequence. Because ULS forest point cloud data often only capture canopy information, the density of the point cloud data it can obtain, especially in the stem part, is significantly lower than that of TLS point clouds. This results in a certain gap in segmentation accuracy compared to bottom-up segmentation methods based on tree trunks (Dersch *et al.* 2021; Jaskierniak *et al.* 2021). Additionally, the DBSCAN-Adjust algorithm requires traversing the points to be clustered one by one, which involves a copious amount of computation and can consume a significant amount of time when processing large-scale point cloud data. Further optimization of computational efficiency is needed in future work.

In areas with poor point cloud quality, when dealing with severe tree crown overlapping or individual trees with large canopies, the climbing algorithm may also exhibit some segmentation errors. This is mainly due to the large distances between some false vertices, which are mistakenly identified as true vertices during the filtering process. Because the climbing algorithm primarily segments trees from the canopy perspective, its accuracy still needs improvement for fallen trees and severely deformed tree crowns in dense forest areas.

Conclusion

Segmenting individual trees in mixed forests within complex environments is a highly challenging task. To address this issue, we developed a new individual tree segmentation algorithm called the "climbing algorithm" and combined it with the PointMLP deep learning model, attempting to achieve more accurate forest extraction and individual tree segmentation. Through a comparative analysis of the PointMLP deep learning model and the PointNet model, we confirmed that PointMLP possesses crucial advantages in extracting forest point clouds in complex environments. The training and testing accuracies of the PointMLP model both reached values greater than 0.9, significantly surpassing those of the PointNet model, which stabilized at an accuracy of around 0.7.

Additionally, the climbing algorithm introduced in this study demonstrated high accuracy and robustness in individual tree segmentation tasks under conditions of tree crown overlap. Particularly in forest areas with uneven tree spacing, the climbing algorithm successfully reduced false positives and false negatives by dynamically adjusting the clustering range. The higher Pr, Re, and F-score of the accuracy results made the climbing algorithm outperforming the PCS algorithm.

Although the climbing algorithm showed low parameter sensitivity in most test data sets, which enhanced its adaptability and efficiency in processing point cloud data in complex forest areas, it still contained segmentation errors upon encountering poor-quality point cloud data, extremely dense forest areas, and individual trees with abnormal crown shapes. This highlights the necessity for algorithm optimization and improved processing efficiency in future research, particularly to find a more balanced solution between computational complexity and accuracy.

Overall, we demonstrated that the combination of the PointMLP deep learning algorithm and the climbing algorithm can effectively extract forest point clouds and segment individual trees in mixed forests within complex environments. Our investigations showcased the advantages of advanced methods in improving classification accuracy and handling individual tree segmentation tasks while also pointing out existing limitations and directions for future improvement. The climbing algorithm framework proposed in this study can achieve high-quality segmentation results in large-scale ULS point cloud data, which provides novel ideas and solutions for individual tree segmentation in forestry surveys and management processes.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant 32361143869). The authors also thank the anonymous reviewers who helped strengthen the paper.

References

- Balsi, M., S. Esposito, P. Fallavollita and C. Nardinocchi. 2018. Single-tree detection in high-density LiDAR data from UAV-based survey. *European Journal of Remote Sensing* 51(1):679–692.
- Dassot, M., T. Constant and M. Fournier. 2011. The use of terrestrial LiDAR technology in forest science: Application fields, benefits and challenges. *Annals of Forest Science* 68:959–974.
- Dersch, S., M. Heurich, N. Krueger and P. Krzystek. 2021. Combining graph-cut clustering with object-based stem detection for tree segmentation in highly dense airborne lidar point clouds. *ISPRS Journal of Photogrammetry and Remote Sensing* 172:207–222.

- Ester, M., H.-P. Kriegel, J. Sander and X. Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. *KDD '96: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 2–4 August 1996, Portland, Oregon. pp. 226–231.
- Eysn, L., M. Hollaus, E. Lindberg, F. Berger, J.-M. Monnet, M. Dalponte, M. Kobal, M. Pellegrini, E. Lingua, D. Mongus and N. Pfeifer. 2015. A benchmark of lidar-based single tree detection methods using heterogeneous forest data from the alpine space. *Forests* 6(5):1721–1747.
- Gougeon, F. A. 1995. A crown-following approach to the automatic delineation of individual tree crowns in high spatial resolution aerial images. *Canadian Journal of Remote Sensing* 21(3):274–284.
- Jaskierniak, D., G. Kuczera, R. Benyon and L. Wallace. 2015. Using tree detection algorithms to predict stand sapwood area, basal area and stocking density in *Eucalyptus regnans* forest. *Remote Sensing* 7(6):7298–7323.
- Jaskierniak, D., A. Lucieer, G. Kuczera, D. Turner, P. N. J. Lane, R. G. Benyon and S. Haydon. 2021. Individual tree detection and crown delineation from unmanned aircraft system (UAS) LiDAR in structurally complex mixed species eucalypt forests. *ISPRS Journal of Photogrammetry and Remote Sensing* 171:171–187.
- Kellner, J. R., J. Armston, M. Birrer, K. C. Cushman, L. Duncanson, C. Eck, C. Fallegger, B. Imbach, K. Král, M. Krůček, J. Trochta, T. Vrška and C. Zraggen. 2019. New opportunities for forest remote sensing through ultra-high-density drone lidar. *Surveys in Geophysics* 40:959–977.
- Koch, B., U. Heyder and H. Weinacker. 2006. Detection of individual tree crowns in airborne lidar data. *Photogrammetric Engineering and Remote Sensing* 72(4):357–363.
- Li, L., Q. Guo, S. Tao, M. Kelly and G. Xu. 2015. Lidar with multi-temporal MODIS provide a means to upscale predictions of forest biomass. *ISPRS Journal of Photogrammetry and Remote Sensing* 102:198–208.
- Li, W., Q. Guo, M. K. Jakubowski and M. Kelly. 2012. A new method for segmenting individual trees from the lidar point cloud. *Photogrammetric Engineering and Remote Sensing* 78(1):75–84.
- Liu, B., H. Huang, X. Tian and M. Ren. 2022. Individual tree species classification using the pointwise MLP-based point cloud deep learning method. *Environmental Sciences Proceedings* 22(1):19.
- Liu, Q., L. Jing, Y. Li, Y. Tang, H. Li and Q. Lin. 2016. A tree canopy height delineation method based on morphological reconstruction—Open crown decomposition. *IOP Conference Series: Earth and Environmental Science* 34:012020.
- Liu, Z., H. Hu, Y. Cao, Z. Zhang and X. Tong. 2020. A closer look at local aggregation operators in point cloud analysis. In *Computer Vision – ECCV 2020*. edited by A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm. Cham, Switzerland: Springer, pp. 326–342.
- Lovell, J.L., D.L.B. Jupp, G.J. Newnham and D.S. Culvenor. 2011. Measuring tree stem diameters using intensity profiles from ground-based scanning lidar from a fixed viewpoint. *ISPRS Journal of Photogrammetry and Remote Sensing* 66(1):46–55.
- Lu, X., Q. Guo, W. Li and J. Flanagan. 2014. A bottom-up approach to segment individual deciduous trees using leaf-off lidar point cloud data. *ISPRS Journal of Photogrammetry and Remote Sensing* 94:1–12.
- Ma, X., C. Qin, H. You, H. Ran and Y. Fu. 2022. Rethinking network design and local geometry in point cloud: A simple residual MLP framework. arXiv:2202:07123. <https://doi.org/10.48550/arXiv.2202.07123>.
- Pirotti, F. 2010. Assessing a template matching approach for tree height and position extraction from lidar-derived canopy height models of *Pinus pinaster* stands. *Forests* 1(4):194–208.
- Qi, C. R., H. Su, K. Mo and L. J. Guibas. 2017. PointNet: Deep learning on point sets for 3D classification and segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 21–26 July 2017, Honolulu, HI, pp. 652–660.
- Schiefer, F., T. Kattenborn, A. Frick, J. Frey, P. Schall, B. Koch and S. Schmidlein. 2020. Mapping forest tree species in high resolution UAV-based RGB-imagery by means of convolutional neural networks. *ISPRS Journal of Photogrammetry and Remote Sensing* 170:205–215.
- Singer, N. M. and V. K. Asari. 2021. DALES objects: A large scale benchmark dataset for instance segmentation in aerial lidar. *IEEE Access*, 9:97495–97504.
- Tao, S., Q. Guo, L. Li, B. Xue, M. Kelly, W. Li, G. Xu and Y. Su. 2014. Airborne lidar-derived volume metrics for aboveground biomass estimation: A comparative assessment for conifer stands. *Agricultural and Forest Meteorology* 198–199:24–32.
- Tao, S., F. Wu, Q. Guo, Y. Wang, W. Li, B. Xue, H. Hu, P. Li, D. Tian, C. Li, H. Yao, Y. Li, G. Xu and J. Fang. 2015. Segmenting tree crowns from terrestrial and mobile LiDAR data by exploring ecological theories. *ISPRS Journal of Photogrammetry and Remote Sensing* 110:66–76.
- Torabzadeh, H., R. Leiterer, A. Hueni, M. E. Schaepman and F. Morsdorf. 2019. Tree species classification in a temperate mixed forest using a combination of imaging spectroscopy and airborne laser scanning. *Agricultural and Forest Meteorology* 279:107744.
- Vauhkonen, J., L. Ene, S. Gupta, J. Heinzel, J. Holmgren, J. Pitkänen, S. Solberg, Y. Wang, H. Weinacker, K.M. Hauglin, V. Lien, P. Packalén, T. Gobakken, B. Koch, E. Næsset, T. Tokola and M. Maltamo. 2012. Comparative testing of single-tree detection algorithms under different types of forest. *Forestry* 85(1):27–40.
- Wallace, L., A. Lucieer and C. S. Watson. 2014. Evaluating tree detection and segmentation routines on very high resolution UAV LiDAR data. *IEEE Transactions on Geoscience and Remote Sensing* 52(12):7619–7628.
- Wang, L., P. Gong and G. S. Biging. 2004. Individual tree-crown delineation and treetop detection in high-spatial-Resolution aerial imagery. *Photogrammetric Engineering and Remote Sensing* 70(3):351–357.
- Weiser, H., J. Schäfer, L. Winiwarter, N. Krašovec, F. E. Fassnacht and B. Höfle. 2022. Individual tree point clouds and tree measurements from multi-platform laser scanning in German forests. *Earth System Science Data* 14(7):2989–3012.
- Windrim, L. and M. Bryson. 2020. Detection, segmentation and model fitting of individual tree stems from airborne laser scanning of forests using deep learning. *Remote Sensing* 12(9):1469.
- Wu, B., B. Yu, W. Yue, S. Shu, W. Tan, C. Hu, Y. Huang, J. Wu and H. Liu. 2013. A voxel-based method for automated identification and morphological parameters estimation of individual street trees from mobile laser scanning data. *Remote Sensing* 5(2):584–611.
- Yin, D. and L. Wang. 2019. Individual mangrove tree measurement using UAV-based LiDAR data: Possibilities and challenges. *Remote Sensing of Environment* 223:34–49.
- Zhao, K., S. Popescu and R. Nelson. 2009. Lidar remote sensing of forest biomass: A scale-invariant estimation approach using airborne lasers. *Remote Sensing of Environment* 113(1):182–196.
- Zhen, Z., L. J. Quackenbush and L. Zhang. 2016. Trends in automatic individual tree crown detection and delineation—Evolution of LiDAR data. *Remote Sensing* 8(4):333.